

Exercise 4 Solution

Part A: Lambda Calculus Computations

Question 1: Lambda Expression Evaluation

Expression 1: $(\lambda z. z) (\lambda y. y y) (\lambda x. x a)$

Perform beta reduction on $(\lambda z. z)$ with argument $(\lambda y. y y)$:

$$[z \mapsto (\lambda y. y y)]z = (\lambda y. y y)$$

Now we have: $(\lambda y. y y) (\lambda x. x a)$

Perform another beta reduction:

$$[y \mapsto (\lambda x. x a)](y y) = (\lambda x. x a) (\lambda x. x a)$$

Perform beta reduction on the left lambda:

$$[x \mapsto (\lambda x. x a)](x a) = (\lambda x. x a) a$$

Perform final reduction:

$$[x \mapsto a](x a) = a a$$

Expression 2: $(\lambda x. \lambda y. x y y) (\lambda a. a) b$

First reduction (substitute $(\lambda a. a)$ for x):

$$(\lambda y. (\lambda a. a) y y) b$$

Internal reduction on $(\lambda a. a)y$:

$$(\lambda y. y y) b$$

Final reduction:

$b\ b$

Expression 3: $((\lambda x. \lambda y. (x\ y)) (\lambda y. y))\ w$

First reduction (substitute $(\lambda y. y)$ for x):

$((\lambda y. ((\lambda y. y)\ y))\ w)$

Perform α -conversion on the internal bound variable to avoid confusion:

$(\lambda y. (\lambda z. z)\ y)\ w$

Internal reduction:

$(\lambda y. y)\ w$

Final reduction:

w

Expression 4: $(\lambda x. y)\ ((\lambda y. y\ y\ y)\ (\lambda x. x\ x\ x))$

Call-By-Value (CBV): In this strategy, we must reduce the argument to a value before substitution. The argument is $((\lambda y. y\ y\ y)\ (\lambda x. x\ x\ x))$. Its reduction will yield $(\lambda x. x\ x\ x)\ (\lambda x. x\ x\ x)\ (\lambda x. x\ x\ x)$, and each additional step will only grow the expression. This is an infinite derivation (divergence), so the expression does not reach a value under CBV.

Call-By-Name (CBN): In this strategy, we substitute the argument as-is without evaluating it first.

$[x \mapsto ((\lambda y. y\ y\ y)\ (\lambda x. x\ x\ x))] y = y$

The result is y .

Did we get the same result? No. In CBN we got y , and in CBV the computation did not terminate.

Question 2: Boolean Expressions

Computing test (or tru fls) a b in CBV:

First, evaluate the argument or tru fls:

$$\begin{aligned}\text{or tru fls} &= (\lambda b. \lambda c. b \text{ tru } c) \text{ tru fls} \rightarrow (\lambda c. \text{tru tru } c) \text{ fls} \rightarrow \text{tru tru fls} \\ \text{tru tru fls} &= (\lambda t. \lambda f. t) \text{ tru fls} \rightarrow (\lambda f. \text{tru}) \text{ fls} \rightarrow \text{tru}\end{aligned}$$

Now substitute into test:

$$\begin{aligned}\text{test tru a b} &= (\lambda l. \lambda m. \lambda n. l \text{ m } n) \text{ tru a b} \\ &\rightarrow (\lambda m. \lambda n. \text{tru m } n) \text{ a b} \\ &\rightarrow (\lambda n. \text{tru a } n) \text{ b} \\ &\rightarrow \text{tru a b}\end{aligned}$$

Final evaluation:

$$\text{tru a b} = (\lambda t. \lambda f. t) \text{ a b} \rightarrow (\lambda f. \text{a}) \text{ b} \rightarrow \text{a}$$

Definition of nand:

$$\text{nand} = \lambda b. \lambda c. b (c \text{ fls } \text{tru}) \text{ tru}$$

Computing nand tru fls:

$$\begin{aligned}\text{nand tru fls} &\rightarrow \text{tru (fls fls tru) tru} \rightarrow (\lambda t. \lambda f. t) (\text{fls fls tru}) \text{ tru} \\ &\rightarrow \text{fls fls tru} \rightarrow (\lambda t. \lambda f. f) \text{ fls tru} \rightarrow \text{tru}\end{aligned}$$

Computing nand tru tru:

$$\begin{aligned}\text{nand tru tru} &\rightarrow \text{tru (tru fls tru) tru} \\ &\rightarrow \text{tru fls tru} \rightarrow (\lambda t. \lambda f. t) \text{ fls tru} \rightarrow \text{fls}\end{aligned}$$

Question 3: Arithmetic Expressions

c₀ succ in CBN: c₀ ignores its first argument (the function) and returns the second.

$$(\lambda s. \lambda z. z) \text{succ} \rightarrow \lambda z. z$$

This is the identity function, which represents c_0 and not c_1 .

c_0 succ in CBV: The result is the same. Since `succ` is a value (an abstraction), it is substituted for `s`, and the result is c_0 .

Definition of `isodd`:

$$\text{isodd} = \lambda m. m (\lambda b. b \text{fls} \text{tru}) \text{fls}$$

Calculations:

Let us define **`not`** = $\lambda b. b \text{fls} \text{tru}$ for clarity.

Calculating `isodd c4`:

$$\text{isodd } c_4 \rightarrow (\lambda m. m \text{not fls}) c_4 \rightarrow c_4 \text{not fls}$$

Substitute the definition of c_4 :

$$\rightarrow (\lambda s. \lambda z. s(s(s(s z)))) \text{not fls}$$

$$\rightarrow (\lambda z. \text{not}(\text{not}(\text{not}(\text{not } z)))) \text{fls}$$

$$\rightarrow \text{not}(\text{not}(\text{not}(\text{not fls})))$$

Reduction steps:

$$1. \text{not fls} \rightarrow (\lambda b. b \text{fls} \text{tru}) \text{fls} \rightarrow \text{fls fls tru} \rightarrow \text{tru}$$

$$2. \text{not tru} \rightarrow (\lambda b. b \text{fls} \text{tru}) \text{tru} \rightarrow \text{tru fls tru} \rightarrow \text{fls}$$

Back to the expression:

$$\text{not}(\text{not}(\text{not}(\text{not fls}))) \rightarrow \text{not}(\text{not}(\text{not tru})) \rightarrow \text{not}(\text{not fls})$$

$$\rightarrow \text{not tru} \rightarrow \text{fls}$$

Result: `fls` (Correct, since 4 is even).

Calculating `isodd c5`:

$$\text{isodd } c_5 \rightarrow (\lambda m. m \text{not fls}) c_5 \rightarrow c_5 \text{not fls}$$

$$\rightarrow (\lambda s. \lambda z. s(s(s(s(s z)))))) \text{not fls}$$

```
→ not(not(not(not(not fls))))
```

Using the reduction logic established above:

```
→ not(not(not(not tru)))
```

```
→ not(not(not fls))
```

```
→ not(not tru)
```

```
→ not fls
```

```
→ tru
```

Result: tru (Correct, since 5 is odd).

Question 4: Y Combinator

Computing fact c_0 :

According to the definition given in the practice, the function uses eq n c_1 as the base case. Since $0 \neq 1$, the code enters the "else" branch where it attempts to calculate times c_0 (fact (pred c_0)).

Since pred c_0 evaluates to c_0 , we get $c_0 \times \text{fact}(c_0)$, which recursively calls fact c_0 again. This leads to an infinite loop. Therefore, the requirement $0! = 1$ is not satisfied.

Fix:

We change the stopping condition to check if the number is c_0 explicitly using iszero.

```
help_fact_fixed = λf. λn. test (iszero n)  $c_1$  (times n (f (pred n)))
```